Performance Testing Phase

Script-Controlled ACL –
Restrict Record Access Based on Field Value

Item	Details
Team ID	**SWTID-2026-9980**
Project	Script-Controlled ACL – Restrict Record Access Based on Field Value
Platform	ServiceNow
Phase	Performance Testing
Version	1.0
Prepared by	Team SWTID-2026-9980
Date	03 October 2026

Contents

1. Introduction and Objective
2. Project Background and ACL Design
3. Scope
4. Test Environment
5. Test Data and Users
6. Test Scenarios
7. Tools
8. Execution Approach
9. Metrics and Acceptance Criteria
10. ACL Script: Slow vs. Optimised
11. Risks and Mitigation
12. Test Schedule
13. Results Table
14. Observations and Recommendations
15. Conclusion
16. Sign-off

1. Introduction and Objective

The Script-Controlled ACL project restricts access to records in ServiceNow based on the value of a field on the record. A scripted Access Control (ACL) evaluates the field value together with the user's role and decides whether the record can be read. Because the ACL script runs for every record returned in a list, report or API call, it can affect system performance as data volume and user count grow.

Objective: Show that the script-controlled ACL does not noticeably slow down list loads, form loads, Table API calls or reports, even with large data volumes (up to 100,000 records) and many concurrent users (up to 100), while still giving the correct access result for every role.

2. Project Background and ACL Design

Item	Details
Requirement	Users without the required role must not see records whose restriction field has the value "restricted"
ACL type / operation	Record ACL, read operation (write follows the same rule if scripted)
Example table / field	Incident table; restriction field u_confidentiality (values: public, restricted)
ACL logic	Allow if user has the privileged role, or the record is not restricted; otherwise deny
Performance risk	The script runs once per row, so queries or heavy logic inside it multiply with record count

The table and field names above are the example configuration used for testing; replace them if your project uses different names.

3. Scope

In scope	Out of scope
Read (and scripted write) ACL on the target table List, form, Table API and report performance Comparison across roles Concurrent user load (10 / 50 / 100)	Functional ACL validation (covered in UAT / ATF) Other tables and unrelated integrations Production load testing

4. Test Environment

Item	Details
Instance	Sub-production performance/test instance (never production)
Configurations	Baseline: script ACL inactive (role-only ACL) With ACL: script ACL active
Data volumes	10,000 / 50,000 / 100,000 records
Constraints	Same data, users and time window for both configurations; no other heavy jobs running

5. Test Data and Users

Records are created with a realistic mix of the restriction field value (about 50% restricted, 50% public), spread across assignment groups and users, using a background script. The same dataset is used for baseline and with-ACL runs.

User	Role	Expected access
perf.restricted	Base role only	Denied on restricted records
perf.allowed	Base role + access role	Allowed on matching records
perf.privileged	Privileged role	Full access
perf.admin	admin	Full access

6. Test Scenarios

ID	Scenario	Steps	Measure
PT-01	List view load (20/50/100 rows)	Open the incident list as restricted, allowed and admin users	Page response time
PT-02	Form open: allowed record	Open a record the user may access	Load time
PT-03	Form open: restricted record	Open a record that should be denied	Time to deny
PT-04	Table API GET	GET /api/now/table/incident with limit and filters	Response time, rows returned
PT-05	Report / dashboard	Run a report on the table	Query time
PT-06	Concurrency	10 / 50 / 100 virtual users running PT-01 and PT-02	Throughput, error rate
PT-07	Role comparison	Repeat PT-01 for each role	Variance by role

7. Tools

Tool	Purpose
Debug Security Rules (Session Debug)	Which ACLs ran and how long each took
Transaction Logs (syslog_transaction)	Response time per request
Script Profiler / Debug Log	Slow code inside the ACL script
Slow Query Log (syslog_db_slow)	Slow GlideRecord queries
stats.do	Instance health, semaphores, sessions
JMeter / Selenium	Concurrent user simulation
Table API client	API-level load
ATF	Confirm ACL logic still works

8. Execution Approach

- Prepare data and users; note the instance state and clear caches before each run.
- Run PT-01 to PT-07 with the script ACL inactive (baseline); repeat each at least 3 times and average.
- Activate the script ACL and run the same scenarios.
- Export transaction log entries and Debug Security Rules output for each run.
- Calculate overhead: (With ACL – Baseline) ÷ Baseline × 100.
- If criteria are not met, optimise the ACL script and retest.

9. Metrics and Acceptance Criteria

Criterion	Target
Average list / form response time	Under 3 seconds
ACL overhead vs. baseline	Under 15%
Error rate at peak load	Under 1%
Slow queries caused by the ACL	None
Response time variance between roles	No unexplained spikes

10. ACL Script: Slow vs. Optimised

The ACL script runs once per row, so queries must be avoided and cheap checks done first. Field and role names are the example configuration.

Slow pattern (initial version)

```
answer = false;
var gr = new GlideRecord('some_other_table');   // query runs for every row
gr.addQuery('user', gs.getUserID());
gr.query();
if (gr.next() && current.u_confidentiality == 'restricted')
    answer = false;
else
    answer = true;
```

Optimised pattern (final version)

```
// 1. Cheap role check first
if (gs.hasRole('privileged_role')) { answer = true; }
// 2. Simple field comparison, no query
else if (current.u_confidentiality != 'restricted') { answer = true; }
// 3. Otherwise deny
else { answer = false; }
```

Optimisation checklist

- Role and simple field checks before any query.
- No GlideRecord queries inside the ACL; cache lookups if unavoidable.
- Index the restriction field the ACL depends on.
- Use ACL condition and role fields so the platform skips the script when possible.
- Remove gs.log / gs.info calls.
- Retest after every change, including access correctness.

11. Risks and Mitigation

Risk	Mitigation
Test instance differs from production	Match data volume and configuration as closely as possible
Other jobs distort results	Schedule tests in a quiet window; repeat runs
ACL script changes during testing	Freeze the script version; retest fully after any change
Unrealistic test data	Use mixed field values and realistic distribution

12. Test Schedule

Activity	Duration	Owner
Test data and user setup	1 day	Admin
Baseline runs (ACL inactive)	1 day	Performance Tester
With-ACL runs	1 day	Performance Tester
Analysis, script optimisation and retest	2 days	Developer / Tester
Report and sign-off	1 day	Test Lead

13. Results Table

Scenario	Users / Records	Baseline (s)	With ACL (s)	Overhead %	Result
PT-01 List load (20 rows)	1 / 10k	0.80	0.86	7.5%	Pass
PT-01 List load (50 rows)	1 / 100k	1.40	1.55	10.7%	Pass
PT-02 Form (allowed)	1 / 100k	0.90	0.95	5.6%	Pass
PT-03 Form (restricted)	1 / 100k	0.90	0.75	-16.7%	Pass
PT-04 Table API	10 / 100k	1.10	1.22	10.9%	Pass
PT-05 Report	1 / 100k	2.10	2.30	9.5%	Pass
PT-06 Concurrency	50 / 100k	1.80	2.00	11.1%	Pass
PT-06 Concurrency	100 / 100k	2.40	2.72	13.3%	Pass

Values are sample results for the optimised ACL script. Replace them with the measured values from your own test runs.

14. Observations and Recommendations

- The initial script, which ran a GlideRecord query inside the ACL, showed about 38% overhead on the 100k-record list load (sample figure) and was flagged in the Slow Query Log.
- Moving the privileged-role check first, replacing the query with a direct field comparison and indexing the restriction field brought the overhead to 10.7%.
- Denied form loads were faster than baseline because less data is rendered for a restricted record.
- Response times were similar for all roles, and the error rate at 100 users was 0.2%.

- Recommendation: deploy the optimised script, keep the field indexed and re-run PT-01 and PT-06 if the record volume grows significantly.

15. Conclusion

With the optimised script, the script-controlled ACL stays within all acceptance criteria: average response time under 3 seconds, overhead under 15%, error rate under 1% and no slow queries. The ACL is ready to move to the next phase.

16. Sign-off

Name	Role	Date	Status
Team SWTID-2026-9980	Test Lead	03 October 2026	Approved
Team SWTID-2026-9980	Developer	03 October 2026	Approved
Team SWTID-2026-9980	Business Owner	03 October 2026	Approved